Intern Project: Multi-Tenant Client Feedback & Support Portal

1. What You're Building (In Plain Terms)

You're building a mini version of tools like Zendesk or Intercom. Different companies ("tenants") sign up, and each one gets its own private space to:

- Handle customer support tickets
- Collect customer feedback
- See stats about how they're doing

Key idea — Multi-tenancy: Imagine three companies use your app: a school, a design agency, and an online store. They all use the *same* app and database, but none of them can ever see each other's data, users, or tickets. That separation is the core technical challenge of this project. Every single database query you write should be scoped to "only this company's data" — this is the #1 thing to get right.

2. Tech Stack

Layer	Technology
Frontend	Next.js
Backend	NestJS (single monolith app, not microservices)
Database	PostgreSQL
ORM	Prisma
Styling	Tailwind CSS

If you haven't used NestJS or Prisma before, budget time in Week 1 to go through their official "getting started" tutorials before writing project code.

3. Who Uses the App (Roles)

Think of these as four different "accounts" with different permissions:

Role	Think of them as...	Can do
Platform Admin	You (the SaaS owner)	See all tenants, suspend/activate them, manage pricing plans, view overall stats
Tenant Owner	The company that signed up	Manage their org's settings, invite/remove teammates, view their reports, change their subscription
Support Agent	An employee of the tenant	View and respond to tickets, change ticket status
Customer	An end-user of the tenant	Submit tickets, submit feedback, see their own ticket history only

Important: A Support Agent at Company A should never be able to see tickets from Company B, and a Customer should never see other customers' tickets — only their own.

4. Build Order (Suggested Phases)

Don't try to build everything at once. Work through these phases in order — each one is testable on its own before moving to the next.

Phase 1 — Foundation

- Set up the NestJS + Prisma + PostgreSQL project skeleton
- Design your database schema (see Section 5 for the entities you'll need)
- Implement **register organization**, **login**, **email verification**
- Implement **JWT auth** with **refresh tokens**
- Implement **forgot password** / **reset password**

Done when: A new company can register, verify their email, log in, and get a working access + refresh token pair.

Phase 2 — Multi-Tenancy & Roles

- Every table that stores tenant data must have a `tenantId` (or `organizationId`) column
- Add middleware/guards so every API request is automatically scoped to the logged-in user's tenant
- Implement **role-based access control (RBAC)**: Platform Admin, Tenant Owner, Support Agent, Customer

- Write guards/decorators like `@Roles('TenantOwner')` to protect routes

Done when: You can prove (with a test) that a user from Tenant A gets a 403/404 when trying to access Tenant B's data, even with a valid token.

Phase 3 — Team & Organization Management

- Company profile page (name, logo upload, timezone)
- Invite teammates by email (they get a link to set a password and join)
- Assign roles to invited members
- Remove team members

Done when: A Tenant Owner can invite someone, that person can accept the invite and log in with the correct role, and the Owner can later remove them.

Phase 4 — Ticketing System

- Create ticket (with title, description, priority)
- Attach files to a ticket
- Assign a ticket to a Support Agent
- Change ticket status through: `Open → In Progress → Waiting for Customer → Resolved → Closed`
- List tickets with **pagination**, **filtering** (by status/priority/assignee), and **search** (by keyword)

Done when: A Customer can create a ticket, an Agent can pick it up, move it through statuses, and both can see the full history.

Phase 5 — Feedback System

- Create feedback forms (custom questions)
- Customers submit ratings/feedback, optionally anonymously
- Feedback categories

Done when: A tenant can build a simple feedback form and customers can submit responses to it.

Phase 6 — Subscription Plans & Feature Gating

- Three plans: **Free**, **Starter**, **Pro** (see table below)
- Enforce limits in code — e.g., block ticket #51 in a month for a Free-plan tenant, block custom form creation for Free-plan tenants
- Build the upgrade/downgrade flow (doesn't need real payment processing — a mock "change plan" button is fine unless told otherwise)

Plan	Team Members	Tickets/Month	Custom Feedback Forms
Free	2	50	0 (not allowed)
Starter	10	500	up to 5
Pro	Unlimited	Unlimited	Unlimited

Done when: Trying to exceed a plan's limit returns a clear error message instead of silently succeeding or crashing.

Phase 7 — Analytics Dashboard

- Tickets created per month (chart)
- Average response time
- Customer satisfaction score (from feedback ratings)
- Open vs. closed ticket counts

Done when: A Tenant Owner can load their dashboard and see accurate numbers that match what's actually in the database.

Phase 8 — Notifications & Audit Logs

- Trigger notifications for: ticket assigned, ticket updated, feedback received, subscription changed (in-app notification list is enough; email is a stretch goal)
- Audit log that records: logins, ticket updates, role changes, subscription changes — who did what, and when

Done when: Every one of the four tracked event types shows up correctly in the audit log with a timestamp and user.

Phase 9 — Platform Admin Tools

- List all tenants with basic stats
- Suspend / reactivate a tenant (suspended tenants can't log in)
- Manage what the available subscription plans are

Done when: Suspending a tenant immediately blocks all of that tenant's users from logging in.

5. Core Database Entities (Starting Point)

You'll refine this as you go, but plan for roughly these tables:

- `Organization` (tenant)
- `User` (with a role + belongs to an Organization, except Platform Admins)
- `Invitation`
- `Ticket`
- `TicketAttachment`
- `FeedbackForm`
- `FeedbackResponse`
- `SubscriptionPlan`
- `AuditLog`
- `Notification`

6. Frontend Pages Checklist

Public (no login required)

- Landing page
- Pricing page
- Login page
- Register page

Dashboard (after login)

- Overview
- Tickets
- Feedback
- Team Members
- Analytics
- Billing
- Settings

7. Definition of Done for the Whole Project

- ☐ Two different companies can use the app at the same time with zero data leakage between them
- ☐ All four roles work as described and can't do things outside their permissions
- ☐ Plan limits (team size, ticket count, form count) are actually enforced, not just displayed
- ☐ Basic tests exist for the tenant-isolation logic (this is the most important thing to test)
- ☐ Analytics numbers are correct, not placeholder/fake data
- ☐ Audit log captures all four required event types

8. Tips

- Get Phase 1 and 2 rock-solid before moving on — almost every later feature depends on tenant isolation working correctly.
- When in doubt about a permission question ("can a Support Agent do X?"), default to the least access and ask rather than guessing wide-open access.
- Ask for a short check-in/demo at the end of each phase rather than waiting until the whole thing is done.